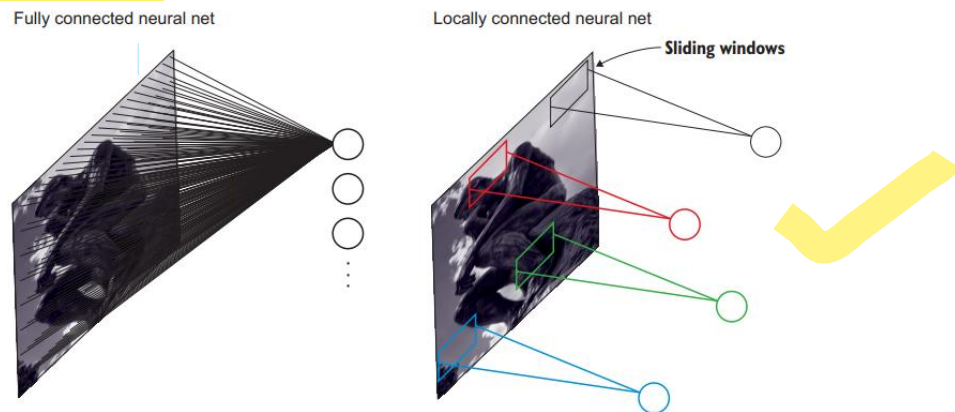


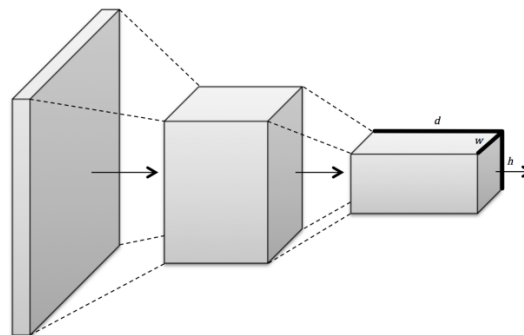
## Convolutional Neural Networks

### Convolutional Neural Networks

- Abbreviated as CNN, also referred to as ConvNet.
- The convolutional network takes advantage of the fact that we're analyzing images, and sensibly constrains the architecture of the deep network so that we drastically reduce the number of parameters in our model.
- The neurons in a convolutional layer are only connected to a small, local region of the preceding layer, avoiding the wastefulness of fully-connected neurons.

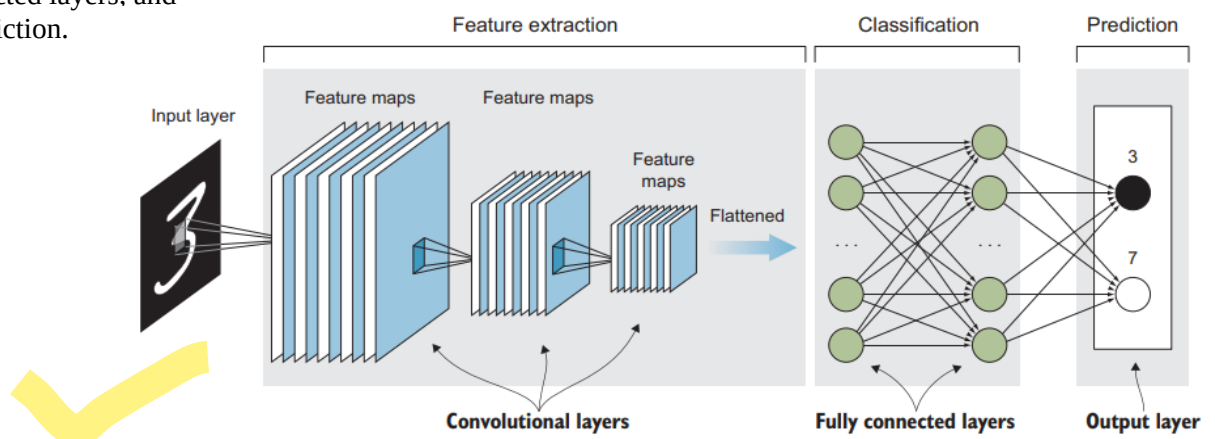


- Layers of a convolutional network have neurons arranged in three dimensions, so layers have a width ( $w$ ), height ( $h$ ), and depth ( $d$ ).



### Convolutional Neural Network Architecture

- The architecture of a CNN is pretty much similar to a vanilla neural network; the difference is that in a CNN we use additional convolutional layers.
- The CNN architecture consists of the following:
  - input layer,
  - convolutional layers (convolution + pooling),
  - fully connected layers, and
  - output prediction.



CNN for Classification of a Handwritten digit as 3 or 7

1. Feed the raw image to the convolutional layers.
2. The image passes through the CNN layers to detect patterns and extract features called feature maps.
  - It is called a feature map because it is a mapping of where a certain kind of feature is found in the image.
  - CNNs look for features such as straight lines, edges, or even objects; whenever they spot these features, they report them to the feature map.
  - Each feature map is looking for something specific: one could be looking for straight lines and another for curves.
  - Notice that the image dimensions shrink after each layer, and the number of feature maps (the layer depth) increases until we have a long array of small features in the last layer of the feature extraction part.
3. The output of this step is then flattened to a vector of the learned features of the image.
4. The flattened feature vector is fed to the fully connected layers to classify the extracted features of the image.

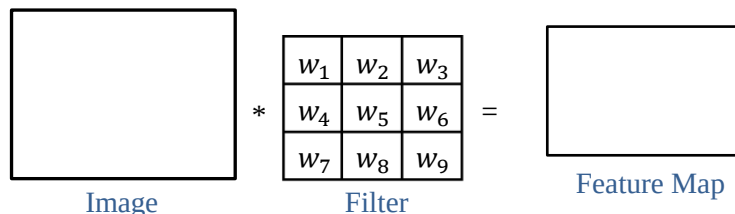
### The Convolutional Layer

- The convolutional layer is the fundamental building block of a CNN.
- It acts as a feature-detection window that slides across the image pixel by pixel to extract meaningful patterns such as edges, textures, shapes, and object parts.
- These extracted patterns help the network identify and distinguish objects in the image.

### The Convolution Operation

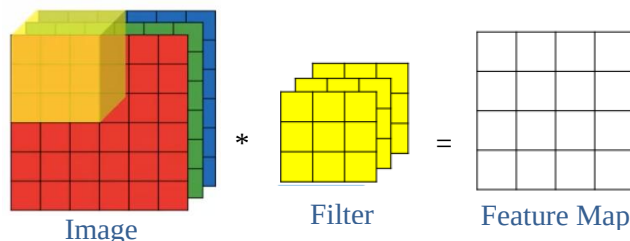
- In mathematics, convolution is an operation between two functions that produces a third transformed function.
- In the context of CNNs, the first function is the multidimensional input image, and the second function is the convolutional filter, also called kernel. This output of this operation is called a feature map or activation map.
- The filter slides across the entire image.
- At each position, corresponding pixel values from the image and filter are multiplied element-wise.
- These products are then summed to produce a single new pixel value in the feature map.
- By repeating this process over all regions of the image, the network transforms the original image into a modified representation that highlights important features.
- $n_h \times n_w$  input, convolved with a  $f_h \times f_w$  filter results in a  $(n_h - f + 1) \times (n_h - f + 1)$  feature map. Here  $n_h$  and  $n_w$  respectively represent the height and width of the input, and  $f_h$  and  $f_w$  respectively represent the height and width of the filter.

For simplicity, in the remainder of this handout we will take square image and square filter with dimensions  $n \times n$  and  $f \times f$  respectively.



### Convolution over Volumes

- For multidimensional images, the convolution is performed over more than one axis (convolution over volumes).
- The depth / number of channels in a filter must be same as the depth / number of channels in the input.
- The depth-wise filters could be same or different as per the requirement.
- $n \times n \times d_{in}$  input where  $d_{in}$  is the depth, convolved with a  $f \times f \times d_{in}$  filter results in a  $(n - f + 1) \times (n - f + 1)$  feature map.
- One specific case where convolution over volume is needed is for an input RGB image. These images have pixels represented as RGB values, and so we require three slices in the input volume (one slice for each color). As a result, feature maps must be able to operate over volumes, not just areas.



### Padding

- Padding (p), often called zero-padding, is addition of zeros around the border of an image.
- Padding preserves the spatial size of the input volume so the input and output width and height are same.
- This way, convolutional layers can be used without necessarily shrinking the height and width of the volumes. This is important for building deeper networks, since, otherwise, the height/width would shrink as we went to deeper layers.
- Another advantage of padding is that all the pixels including the ones at the edges are covered in multiple convolutions. Without padding the pixels at the outer edges are used only once.
- Python Tensorflow provides the following two main options:
  - **Valid Convolution:** Uses no padding.  $n \times n \times d_{in}$  input convolved with a  $f \times f \times d_{in}$  filter results in a  $(n - f + 1) \times (n - f + 1)$  feature map.
  - **Same Convolution:** Pad so that output size is the same as the input size.  $n \times n \times d_{in}$  input with padding p, convolved with a  $f \times f \times d_{in}$  filter results in a  $(n + 2p - f + 1) \times (n + 2p - f + 1)$  feature map.

### Stride

- Stride (s) is the amount by which the filter slides over the image.
- Jumping pixels produces smaller output volumes spatially.
- $n \times n \times d_{in}$  input with padding p and stride s, convolved with a  $f \times f \times d_{in}$  filter results in a  $\left(\left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor\right) \times \left(\left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor\right)$  feature map.
- Smaller strides preserve more spatial detail but increase computational cost, whereas larger strides improve efficiency at the expense of feature resolution.
- Strides above 2 are rarely used in modern CNN architectures.

### Number of Filters or Kernels

- Convolutional neural networks do not learn a single filter; instead, they learn multiple filters in parallel for a given input.
- A convolutional layer (which consists of a set of filters) converts one input volume of values into another output volume of values.
- The depth (number of channels) of each filter equals the depth of the input volume, allowing the filter to combine information across all input channels.
- The depth of the output volume equals the number of filters in that layer, because each filter produces one feature map (one output slice).
- Each convolutional layer has one or more learnable filters/kernels.
- Therefore, the number of filters in a layer determines the depth of its output volume.
- In CNNs, convolutional layers act as the hidden layers for feature extraction, and increasing the number of filters increases the representational capacity of the network.
- Similar to dense neural networks, more filters increase model complexity and enable learning more diverse and complex patterns.
- $n \times n \times d_{in}$  input with padding p and stride s, convolved with  $d_{out}$  filters of dimension  $f \times f \times d_{in}$  results in a  $\left(\left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor\right) \times \left(\left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor\right) \times d_{out}$  feature map.

### Filter Size or Kernel size

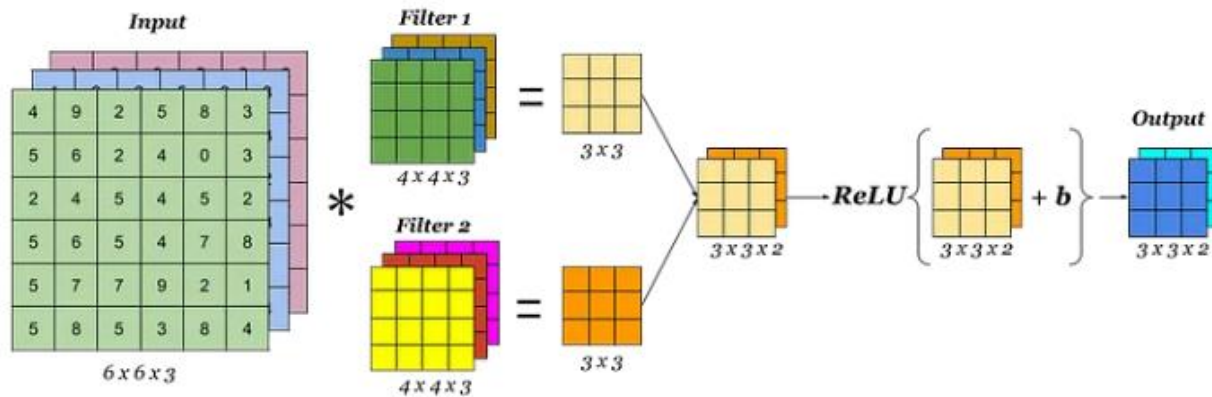
- The filter (kernel) size refers to the spatial dimensions of the convolution kernel (height  $\times$  width).
- It is a matrix of learnable weights that slides over the image to extract features.
- ✓ Smaller filters capture fine local details such as edges, corners, and textures.
- ✓ Larger filters capture broader spatial patterns, but may lose very fine details.
- ✓ Kernel sizes are usually square, with  $3 \times 3$  being most common, although  $1 \times 1$ ,  $5 \times 5$ , and larger kernels are also used in some architectures.

### Activation Function

- After convolution, the linear output is passed through a nonlinear activation function.
- A neuron in the feature map becomes strongly activated when the corresponding filter detects its learned feature at that spatial location.
- This stage is sometimes called the detector stage, since it indicates the presence of specific learned patterns.

If the input pattern matches what the filter has learned  $\rightarrow$   
output value is high  
If it doesn't match  $\rightarrow$  output is low or near zero

- Typically, one bias term is associated with each filter, so the number of biases equals the number of filters (or feature maps).
- The number of parameters is independent of the input image size, which is one major advantage of CNNs over fully connected layers.
- Generally, ReLU function or Tanh function is used as an activation function in a convolution layer.



### Number of Parameters

- Number of parameters is independent of the size of the image.
- Total number of parameters in layer =  
number of filters/kernels  $\times$  filter (height  $\times$  width  $\times$  depth) + number of filters (for biases)

### The Pooling Layer

- A pooling layer (also called subsampling) reduces the spatial dimensions of the feature maps by replacing values in a local region with a summary statistic, such as the maximum or average value.
- Similar to convolution, pooling uses a sliding window (kernel) with a defined size and stride that moves across the feature map. However, pooling kernels contain *no learnable weights*. This layer does not use padding.
- The pooling operation is applied independently to each depth slice (channel) of the input, so it only reduces the height and width, while the number of channels remains unchanged.
- In practice, pooling layers are commonly inserted after one or two convolutional layers in a CNN architecture.
- The primary purpose of pooling is downsampling, i.e., reducing the size of the feature maps.
- By reducing spatial dimensions, pooling decreases the number of parameters and computations passed to subsequent layers, improving computational efficiency.
- As CNNs become deeper with more convolutional layers, the number of parameters and required operations increases significantly. Pooling helps manage this by progressively shrinking the feature representation.
- This reduction in representation size not only lowers time and memory complexity, but also helps control overfitting by forcing the network to retain only the most important features.
- There are two main types of pooling layers: max pooling and average pooling.
  - Max Pooling:** Selects the max pixel value to pass along to the next layer, ignoring the remaining values.
  - Average Pooling:** Average pooling computes the average pixel value to pass along to the next layer.
- A pooling layer filter  $f$  size  $f \times f \times d_{in}$  with stride  $s$ , converts an input of size  $n \times n \times d_{in}$  to an output of size  $\left\lfloor \frac{n-f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n-f}{s} + 1 \right\rfloor \times d_{in}$ .

#### Example:

A pooling filter with a size of  $2 \times 2$  and strides of 2 will reduce the feature map size from  $4 \times 4$  down to  $2 \times 2$ .

|   |   |   |   |
|---|---|---|---|
| 1 | 9 | 6 | 4 |
| 5 | 4 | 7 | 8 |
| 5 | 1 | 2 | 9 |
| 6 | 7 | 6 | 0 |

|   |   |
|---|---|
| 9 | 8 |
| 7 | 9 |

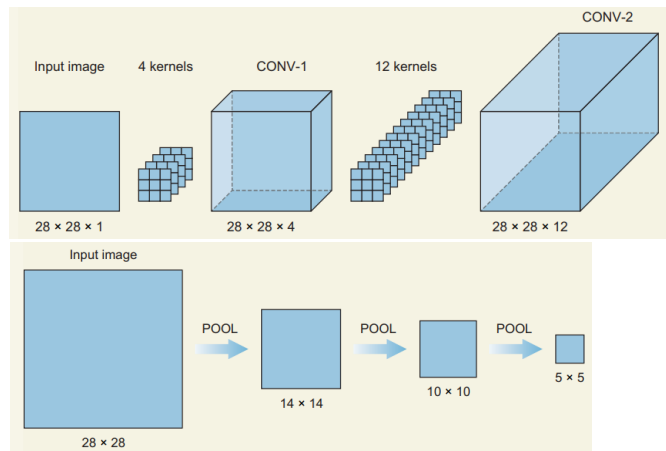
  

|      |      |
|------|------|
| 4.75 | 6.25 |
| 4.75 | 4.25 |

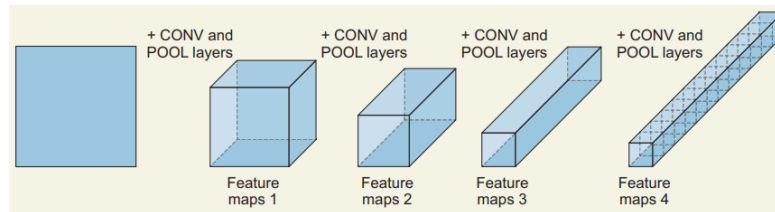
- Filters size or kernel size, stride and pooling type are the hyperparameters associated with this layer.
- Only two major variations of the pooling layer are used.
  - The first is the non-overlapping pooling layer with kernel dimension  $f=2$  and stride  $s=2$ .
  - The second is the overlapping pooling layer with kernel dimension  $f=3$  and stride  $s=2$ .
- Pooling sizes with larger receptive fields are too destructive.
- Max pooling has almost become the default nowadays.

## Cascading Convolutional and Pooling Layers

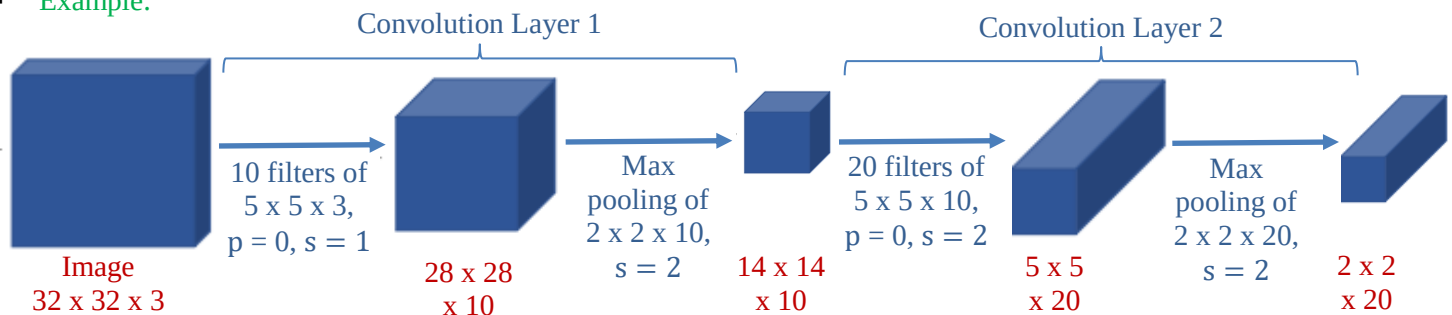
- As stated earlier, it is a common practice to add pooling layers after every one or two convolutional layers in the CNN architecture.
- After the convolutional layers, the image usually keeps its width and height dimensions, but it gets deeper and deeper after each layer.  
true when padding is used for w and h. for d each convolutional layer uses multiple filters
- After the pooling layers, the image keeps its depth but shrinks in width and height.



- Putting the convolutional and pooling together, we get something like this:

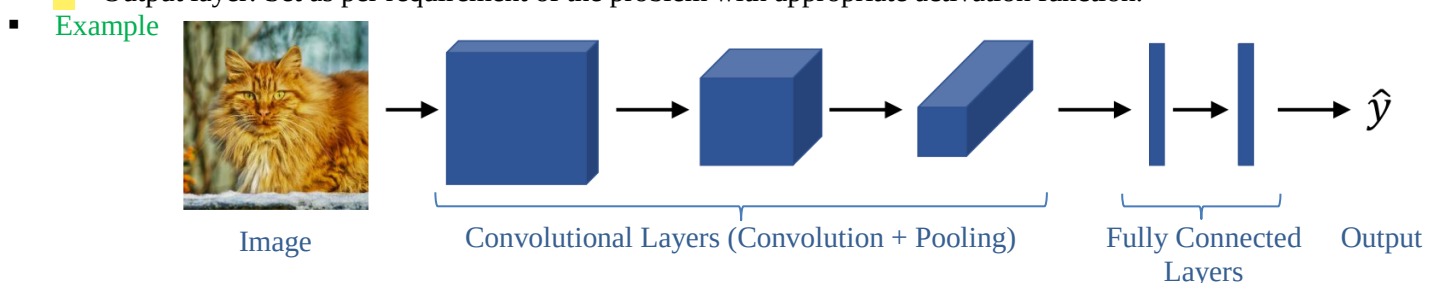


- This keeps happening until we have, at the end, a long tube of small shaped images that contain all the features in the original image.
- The output of the convolutional and pooling layers produces a feature tube that is almost ready to be classified.
- Example:**



## The Fully connected Layer

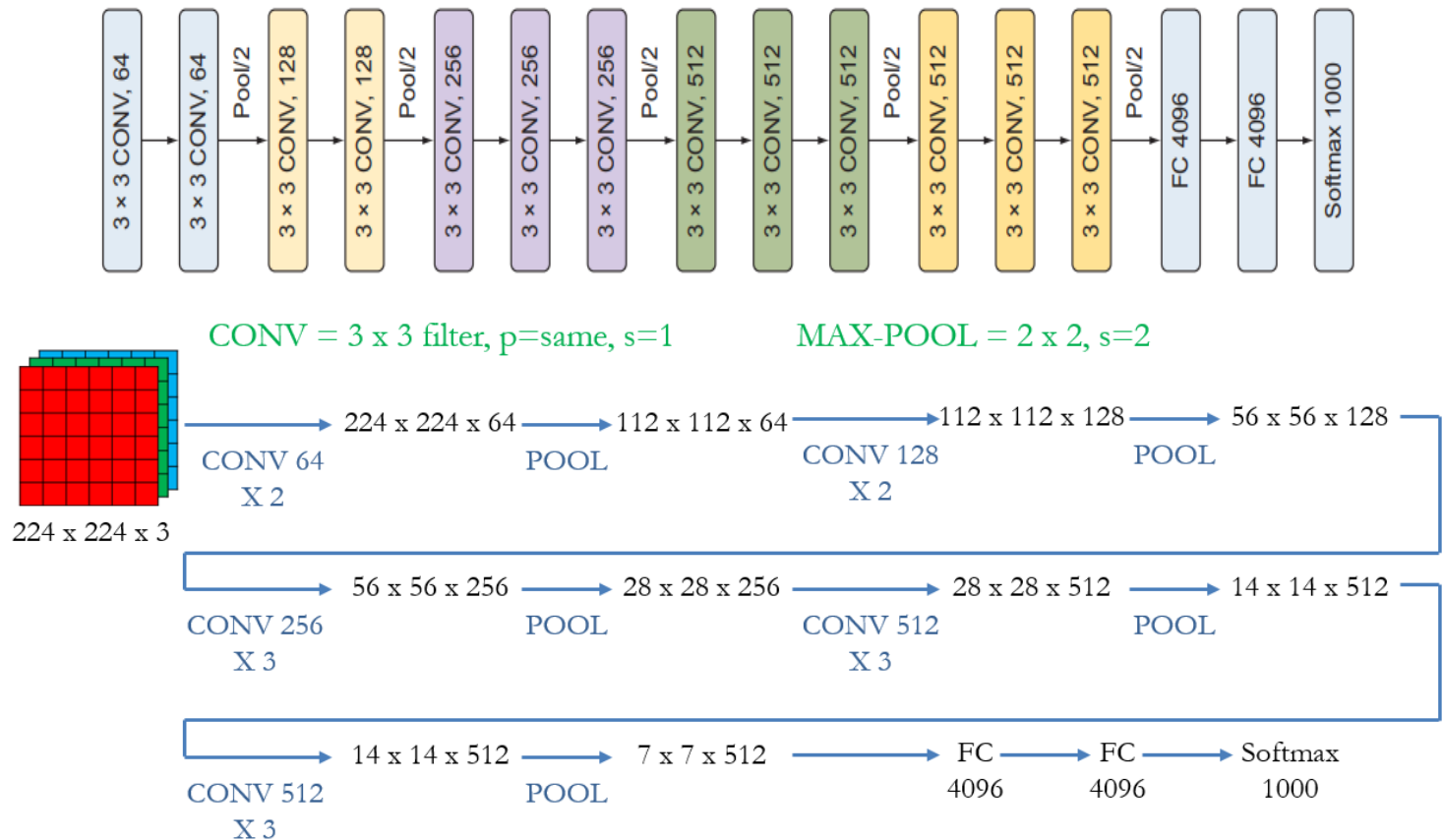
- After passing the image through the feature-learning process using convolutional and pooling layers, we have extracted all the features and put them in a long tube.
- These extracted features are used to classify images, using the regular neural network fully connected (FC) layers.
- Hence the FC layers are same as follows:
  - Input flattened vector: To feed the features tube to the neural network for classification, flatten it to a vector with the dimensions (1, n).
  - Hidden layer: Add one or more fully connected layers, with each layer having one or more neurons.
  - Output layer: Set as per requirement of the problem with appropriate activation function.



### Case Study: Network Architecture of VGG Net

- VGGNet was developed in 2014 by the Visual Geometry Group at Oxford University, hence the name VGG.
- Its uniform architecture makes it appealing in the deep learning community because it is very easy to understand.
- Two popular versions of VGGNet are VGG16 and VGG19. Both are similar, only different in number of layers.
- Research Paper: Karen Simonyan and Andrew Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," 2014.

#### Architecture of VGG16



- Activation function: ReLU
- Regularization: Dropout for the first two fully connected layers, with a dropout ratio set to 0.5. Also used L2 regularization with weight decay of  $5 \times 10^{-4}$ .
- Learning Rate: The learning rate is initially set to 0.01 and then decreased by a factor of 10 when the validation set accuracy stops improving.
- Optimizer: Mini-batch gradient descent with momentum = 0.9.
- Batch size: 256
- Epochs: 74
- Training set: ImageNet (ImageNet Large Scale Visual Recognition Challenge (ILSVRC) 2014)
- VGG16 achieved a top-5 error rate of 8.1% on the ImageNet dataset for image classification, compared to 15.3% achieved by AlexNet.
- VGG19 did even better; it was able to achieve a top-5 error rate of ~7.4%.